

Using Frameworks: Write Up

ICT 4305: Object Oriented Methods and Programming

for

Master of Science

Information Technology

Kalika Browder

University of Denver College of Professional Studies

November 11, 2025

Faculty: Nirav Shah, M.S. , MBA

Director: Cathie Wilson, M.S.

Dean: Bobbie Kite, PhD

Reflection

This assignment did a great job for me of reinforcing the importance of stable identifiers and loose coupling between classes. By implementing the “equals” and “hashCode” parts of the assignment, it made it clear to me that each class that was used in collections should expose a stable key. For example, “customerId” for Customer, “license” for Car, and “permitId” for Permit. Choosing these fields was straightforward for me due to the fact that they are immutable and logically identify instances across the system. For “ParkingCharge”, I wanted to compare all fields so that two charges with the same permit, lot, time, and amount were equal. It was a conservative approach, but allowed me to avoid false merges when a different timestamp or amount existed.

By adding “getCustomerIds()”, “getPermitIds()”, and “getPermitIds(Customer)” I clarified the ownership and reduced the need overall for external classes to access the internal collections directly. These accessor methods preserved encapsulation by returning unmodifiable collections and preventing clients from mutating internal lists. My idea was that this approach would encourage a design where “ParkingOffice” controls the canonical data, whereas other components consume the derived ID lists.

Throughout this entire revisit to the Parking system, I think the easiest bit for me was the implementation of the three accessor methods. This is because they are simple stream operations, and dramatically improve the API’s expressiveness throughout the system. What was more hard for me was deciding the equality semantics for classes like “ParkingCharge” and “ParkingLot”. For “ParkingLot”, a unique lotId did a decent job. For “ParkingCharge”, I noticed

that a richer equality made more sense because the charges are domain events and two charges that match all significant fields in the system should be considered identical.

This process was aided by my walking through typical workflows, like that of registering the customer, registering the car, issuing the permit, parking, and then charging the customer account. I also reviewed UML diagrams, which helped make it obvious to me which fields were unique identifiers. I made sure to include my unit tests, which were absolutely invaluable in this process, too. Writing tests early and thinking about edge cases, like null permits and customers with no cars, allowed me to catch corner cases, as well.

Unfortunately, I did have to revisit this assignment many times, and lastly after the assignment was already due. After working on my Portfolio assignment for the course, I realized where I had been lacking in previous assignments and where I was losing points - I realized that I had missed including JUnit test classes for all of my main Java classes. I originally had created one or two main tests but did not follow the guideline of “one test class per source class”. This oversight taught me that writing complete test coverage is not just a grading requirement, but it is a professional practice that absolutely ensures that my system behaves correctly when small changes are made later.

In revisiting my code, I created dedicated test files for each class like tht of `ParkingOfficeTest`, `CustomerTest`, `CarTest`, and `PermitTest`. Each test targeted specific methods like “`getCustomerIds()`” and “`equals()/hashCode()`”. Writing these new tests gave me more confidence in my program’s correctness and consistency. It also helped me spot smaller design issues. For example, a case where “`equals()`” depended on mutable data. However, once I fixed

those, my classes started to behave more predictably inside Java collections like List and HashMap.

This experience was the strongest reminder that testing validates design decisions and implementing equality and encapsulation is only meaningful if the tests can confirm they hold up under different inputs. I learned to treat test writing as part of the design phase, rather than an afterthought.

I do wish my original code for the Parking system made it clear whether permit objects are replaced or reused. For example, if permits are mutable or revoked, equality by id remains correct, but the key is to avoid using mutable fields in equals/hashCode. I wasn't necessarily considering expanding on my original code when I created it, which was obviously a misstep, so the overall lesson for me throughout this variation was considering where systems grow and how to expand on them.

A lot of my implementation decisions also revolved around things like whether or not to use immutable IDs for equals/hashCode, returning Collections.unmodifiableList(...) from the ID accessors to preserve encapsulation, using streams and Objects::nonNull to filter. These choices that I ended up making made sense to me because these choices favor readability, safety, and loose coupling.

I am overall proud of my final rework... it was a challenging week for me to figure out how to expand on original assignments and figure out where I had made mistakes, but I do believe that the changes I made overall were beneficial to my system.